By placing the `unsigned` keyword in front of the integer variable declaration, the variable `maxThreshold` may now only store positive values in the range of 0 to 65,535. Table 3-2 provides a list of unsigned variable data types and their modified ranges.

Table 3-2. *Unsigned Data Types*

Name	Size	Range
unsigned char	8 bit	0 to 255
unsigned int	16 bit	0 to 65,535
unsigned long	32 bit	0 to 4,294,967,295

You will notice that an unsigned char has the same value range as the byte although, for this range of values, byte is the preferred data type. Also keep in mind that unsigned data types will roll over just like their signed brethren when they exceed their minimum or maximum values.

In addition to unsigned and const, there are other variable qualifiers that we will revisit in later chapters, as they are needed.

Predefined Constants

In addition to user defined constant variables, the Arduino development environment uses its own predefined constants. The first of these constants defines the Boolean logic states based on logical truth tables using two states: true and false. The `false` condition is easily defined as being 0 or sometimes off. On the other hand, `true` is, in many ways, anything other than false. In this sense, `true` could be 1, or 'on,' but it could also be 5, -42, or even 64,000. Unlike our other constants, `true` and `false` always appear in lowercase.

Our other pairs of constants, which you have already seen some of, are generally used as part of the Arduino's digital I/O capabilities and will be discussed in more depth in later. These constants define the operation of the digital pins as either `INPUT` or `OUTPUT` using the `pinMode()` function, or define the pin levels as being `HIGH` or `LOW`, roughly equivalent to pin states of +5 volts and 0 volts, or on and off respectively.

Variable Scope

While we have so far been discussing how variables are declared, where we declare variables is as important as how we declare them. The location of a variable declaration in the larger sketch determines where that variable can be used, or what is known as **variable scope**.

```
const int red = 9;
```

This variable from our previous example is declared outside the `setup()` and `loop()` functions at the beginning of the source code. This is known as a **global variable** and can be used by all functions in the sketch regardless of location. If, however, this variable was declared inside the `setup()` function, it would become a **local variable** and would be unavailable to `loop()` or any other functions. Local variables can also be used inside `for` loops, commonly as counters, where only that statement can use it and not the rest of the function. In fact, it is entirely possible for one variable name to be used in two different